Catálogo de Parámetros UCO Lab

Guía de integración para los equipos del semillero

Versión 1.0 · 3 de noviembre de 2026

Para quién es: equipos de Mensajes, Asistencia, Seguridad y demás aplicaciones que necesitan consultar o administrar parámetros del catálogo.

Qué vas a lograr: registrar la configuración de tu aplicación, consultar sus valores por ambiente desde tu backend y montar la administración dentro de tu pantalla.

Cómo leerla: sigue los pasos en orden. Los apartados de comprobación indican el resultado esperado antes de continuar. Las direcciones localhost corresponden al laboratorio; solicita al operador las URL reales para otro despliegue.

Referencia de formato: Mensajes/Guia-integracion-MessageUcoLab.pdf, versión 4.0. Esta guía adapta su recorrido de preparación, seguridad, datos, consumo y verificación a las funciones implementadas en Parámetros.

Contenido

- Guía de integración para los equipos del semillero
- 0. Permitir la pantalla de Parámetros en Seguridad
- Archivos de esta entrega y carga masiva en Seguridad
- Recorrido de integración
- Ejemplo que seguiremos
- 1. Qué se integra
- 2. Requisitos y responsabilidades
- 3. Identificadores y credenciales: no confundirlos
- 4. Preparar los datos en el catálogo
- 5. Generar y entregar la credencial de consulta

- 6. Configurar el backend consumidor
- 7. Autorizar la administración en Seguridad
- 8. Sesión, dominios, HTTPS y CORS
- 9. Publicar o consumir el paquete del microfrontend
- 10. Montar el componente
- 11. Configurar el proxy y los eventos SSE
- 12. Vigencia, estados y fechas
- 13. Verificación antes de entregar la integración
- 14. Problemas frecuentes
- 15. Referencias y prueba local rápida
- Anexo A. Levantar el proveedor en este workspace
- Anexo B. Rutas del microfrontend y consulta
- Anexo C. Leer el valor en tu lógica de negocio
- Anexo D. Lista de entrega del equipo consumidor
- Anexo E. Referencia de cuerpos y comportamiento HTTP
- Anexo F. Inventario completo de las 93 operaciones de negocio
- Anexo G. Rutas técnicas del despliegue

0. Permitir la pantalla de Parámetros en Seguridad

El inicio de sesión de Seguridad solo regresa a las direcciones autorizadas. Antes de registrar recursos o abrir la administración, agregar la pantalla de Parámetros (http://localhost:4200 en este laboratorio). Si el microfrontend se abre dentro de otra aplicación, agregar también el origen de esa aplicación; el ejemplo usa http://localhost:4300.

1. Abrir la carpeta donde se levantó el componente de Seguridad, la que contiene compose.yaml y .env, por ejemplo Seguridad/security-baseline-local.

2. Abrir el archivo .env con un editor de texto.

3. Buscar las siguientes variables. Si no existen, agregarlas al final. Conservar los orígenes que ya tengan y añadir los de Parámetros y del host consumidor separados por comas:

```
PDP_SECURITY_CORS_ALLOWED_ORIGINS=http://localhost:5173,http://localhost:4200,http://localhost:4300
PDP_LOGIN_ALLOWED_RETURN_ORIGINS=http://localhost:5173,http://localhost:4200,http://localhost:4300
```

5173 corresponde a la pantalla de Seguridad; 4200, a la consola de Parámetros; 4300, al host consumidor de ejemplo. Si la aplicación consumidora usa otra dirección, sustituir 4300 por su origen real. Si Mensajes ya está autorizado en http://localhost:5174, conservar también ese origen en ambas listas. No añadir rutas como /parametros al origen: usar esquema, dominio y puerto. localhost y 127.0.0.1 son orígenes diferentes.

Comprobar que Compose utiliza esas variables

En la copia de Seguridad/security-baseline-local/compose.yaml de este workspace, las dos variables aparecen con valores escritos directamente en el servicio pdp. En esa configuración editar solo el .env no cambia los valores del contenedor. Quien opera Seguridad debe hacer que esas entradas de environment lean el .env:

```
services:
  pdp:
    environment:
      PDP_SECURITY_CORS_ALLOWED_ORIGINS: ${PDP_SECURITY_CORS_ALLOWED_ORIGINS}
      PDP_LOGIN_ALLOWED_RETURN_ORIGINS: ${PDP_LOGIN_ALLOWED_RETURN_ORIGINS}
```

El bloque muestra únicamente las entradas que se deben ajustar: conservar el resto de la configuración del servicio. Si el Compose de tu instalación ya usa esta interpolación, basta con actualizar su .env.

4. Guardar los archivos y, en una terminal dentro de security-baseline-local, aplicar la configuración:

```
docker compose up –d pdp
```

5. Abrir http://localhost:4200, iniciar sesión desde el flujo de la aplicación y comprobar que Seguridad permite regresar a la pantalla. Repetir desde http://localhost:4300 si se usa ese host consumidor. Si aparece El destino de retorno no está autorizado, comprobar que el origen exacto esté en PDP_LOGIN_ALLOWED_RETURN_ORIGINS y que el servicio haya recibido el cambio.

Permitir también las peticiones al API de Parámetros

Las variables anteriores pertenecen a Seguridad. Para que el navegador consumidor llame al API del catálogo con credenciales, configurar además esta variable en el backend de Parámetros:

```
FRONTEND_ALLOWED_ORIGINS=http://localhost:4200,http://localhost:4300
```

Conservar los demás orígenes autorizados y aplicar el cambio al despliegue del catálogo. El permiso de retorno del login, CORS del API y los permisos de usuario son configuraciones independientes: este paso permite completar la autenticación y las peticiones del navegador; después se cargan los recursos y se asignan los roles descritos a continuación.

Archivos de esta entrega y carga masiva en Seguridad

• Guia-integracion-Parametros.pdf: guía para compartir con los equipos.

• Guia-integracion-Parametros.md: fuente editable del mismo documento.

• recursos-parametros.xlsx: 92 recursos protegidos, consola central y administración embebida.

• recursos-parametros-microfrontend.xlsx: 17 recursos del widget, como alternativa si solo necesitas esa administración. Sus filas ya están incluidas en el archivo de 92; elige uno de los dos para evitar duplicados.

• Endpoints-Parametros.csv: 93 operaciones de negocio, incluida la lectura por token, con su autenticación y su indicación de carga en Seguridad.

• Parametros.postman_collection.json: las 93 solicitudes para importar en Postman. No ejecutarlas todas como un flujo: incluye eliminaciones y conexiones SSE que permanecen abiertas.

Registrar el servicio, cargar recursos y conceder permisos

1. Iniciar sesión en la pantalla de Seguridad, localmente http://localhost:5173.

2. En Aplicaciones, registrar o seleccionar Catalogo, con URL base http://localhost:8080 en local. Es el nombre predeterminado de SECURITY_APPLICATION_NAME; si el despliegue usa otro, cambiar ese nombre en las filas del Excel. Elegir el ambiente que usa SECURITY_ENVIRONMENT (el código tiene dev como predeterminado; el ejemplo de laboratorio puede usar local). Ambos lados deben coincidir.

3. Guardar la credencial técnica como PEP_REGISTRATION_TOKEN en el backend proveedor. Es distinta del secreto de consulta que recibirá el backend consumidor.

4. En Recursos → Carga masiva, seleccionar recursos-parametros.xlsx. Conserva los encabezados exactos de la plantilla de Seguridad: Nombre de aplicación, Ruta, Método HTTP, y la hoja Recursos. Cada fila identifica una combinación de método y ruta, sin host ni query string.

5. Pulsar Validar archivo. Corregir los errores reportados (nombre de aplicación, autorización del importador o recursos existentes) y volver a validar. Confirmar la carga cuando no haya errores. No cargar otra vez las filas ya registradas.

6. Crear un rol, asociar los recursos que necesite y asignarlo al usuario en la aplicación y ambiente adecuados; también puede usarse un perfil según el modelo de Seguridad. Importar endpoints por sí solo no concede acceso.

7. Verificar una lectura, una escritura y el SSE con ese usuario. Para un rol de lectura conceder GET; para un administrador añadir únicamente las escrituras requeridas.

Los segmentos {id}, {idAplicacion} son variables de ruta, como en los controladores. Las cinco opciones de {recurso} del controlador administrativo están expandidas en el Excel: no hay una ruta genérica que permita cualquier texto. Conservar la sintaxis de variables admitida por el componente de Seguridad; si se registran UUID concretos para restringir una aplicación, sustituir solo el UUID de aplicación y comprobar el resultado con el usuario autorizado y otro sin permisos.

La ruta GET /catalogo-parametros/api/v1/tokens/{id}/parametros está excluida del PEP y no se carga en este Excel. El catálogo exige su propio Bearer, token activo y no vencido. No es lectura anónima. Las 76 operaciones REST generales, contando esta lectura, más las 17 administrativas suman 93; las protegidas suman 92. La carga masiva aquí registra permisos en Seguridad; no importa datos de parámetros. El catálogo actual no tiene un endpoint de importación masiva de parámetros.

Configurar Postman

Importar la colección y definir baseUrl como host, por ejemplo http://localhost:8080, sin agregar el prefijo del API. Completar los UUID de variables con registros reales. Para administrar, usar una sesión de Seguridad válida y la configuración de cookies/autenticación del despliegue. Para consultar por token completar idToken y secretoConsulta como valores privados locales, sin exportarlos. Solo esa solicitud tiene Bearer del catálogo. Las escrituras no capturan automáticamente los IDs: consultar tras crear y completar las variables antes de continuar. Las fechas de ejemplo deben ajustarse a una vigencia válida.

Recorrido de integración

1. Identificar los dos flujos y reunir los requisitos.

2. Preparar Seguridad, los orígenes y el despliegue del proveedor (anexo A si lo ejecutarás localmente).

3. Registrar organización, aplicación, jerarquía y valores por ambiente.

4. Generar la credencial y configurar el cliente del backend.

5. Conceder permisos administrativos y montar el microfrontend.

6. Verificar consulta, aislamiento por ambiente y sincronización.

Ejemplo que seguiremos

La aplicación Inventario UCO necesita el parámetro configuracion-inventario, en una funcionalidad del módulo Inventario. El valor es un objeto JSON como {"maximoProductos":100} para Desarrollo y {"maximoProductos":1000} para Producción. Crear un metadato por ambiente y seleccionar un tipo de metadato compatible con JSON. Los nombres son ejemplos: los UUID y los tipos se obtienen del catálogo de tu instalación.

El backend de Inventario consulta el valor y lo aplica en su lógica. Si su pantalla necesita conocer el máximo, se lo pide al backend de Inventario. La pantalla puede abrir la administración embebida con una sesión autorizada.

Esta guía está dirigida al equipo que integra el catálogo en una aplicación propia. Cubre la preparación del catálogo, la consulta desde el backend consumidor y la administración embebida mediante el microfrontend Gestión de parámetros.

1. Qué se integra

Existen dos flujos con autenticación distinta:

Necesidad • Recorrido • Autenticación

Consultar los parámetros de la aplicación • Frontend consumidor → backend consumidor → API del catálogo • El backend consumidor usa ID de token y secreto de consulta

Administrar parámetros dentro de la aplicación • Frontend consumidor → microfrontend → API del catálogo • Sesión del usuario en Seguridad y permisos sobre las rutas administrativas

Ambos trabajan con la misma base de datos central. El microfrontend incluye sus formularios, validaciones de presentación y estilos; las reglas y la persistencia permanecen en el backend del catálogo. No se replica la base de datos ni se copian sus CRUD a la aplicación consumidora.

El secreto de consulta no autoriza administración. La sesión del usuario no sustituye la credencial que necesita el backend para consultar por token.

2. Requisitos y responsabilidades

El equipo que opera el catálogo debe proporcionar

• Backend del catálogo operativo y con acceso a sus dependencias, incluida Seguridad/PEP.

• Frontend del catálogo o alojamiento del paquete del microfrontend.

• URLs del API, del paquete ESM y del inicio de sesión de Seguridad para cada entorno.

• Un administrador autorizado para registrar aplicaciones, preparar los catálogos y conceder permisos.

• Configuración de orígenes permitidos, sesión, cookies y retornos de autenticación para el dominio consumidor.

Si se despliega también el proveedor, su configuración de SurrealDB, Redis, secretos y Seguridad pertenece al despliegue del catálogo; consultar el README del backend. Esas credenciales de infraestructura no se entregan al consumidor.

El equipo consumidor necesita

• Un backend con cliente HTTP capaz de enviar Authorization: Bearer y leer JSON. Puede usar Spring u otra tecnología.

• Un frontend con un elemento HTML donde montar el componente y control de su ciclo de vida. No tiene que ser Angular: el paquete ESM incluye Angular compilado.

• Navegador con módulos ESM, Shadow DOM y EventSource/SSE.

• Acceso de red al API desde su backend y al paquete y API administrativos desde el navegador o su proxy.

• Un lugar privado para configurar el secreto: variables de entorno o gestor de secretos del backend.

Para ejecutar los ejemplos del repositorio: Docker Desktop con contenedores Linux, o Java 21+ y Maven para BackPrueba, y Node compatible con el Angular del ejemplo para FrontPrueba (22.22.3+ dentro de la rama 22; consultar su package.json). Usar el ESM ya publicado no exige instalar Java, Maven ni Angular en el navegador consumidor.

3. Identificadores y credenciales: no confundirlos

Dato • Lo obtiene de • Dónde se utiliza

UUID de aplicación del catálogo • Aplicaciones en el frontend de Parámetros • applicationId del microfrontend y configuración del backend consumidor

UUID del ambiente • Ambientes del catálogo • Metadatos, asociación parámetro–ambiente y creación del token

ID del token de consulta • Pantalla Tokens, después de crearlo • Ruta de consulta del backend consumidor

Secreto del token de consulta • Se genera en el frontend del catálogo; se entrega al crearlo • Solo en el backend consumidor, como Bearer

ID de aplicación y credencial PEP de Seguridad • Administración de Seguridad • Configuración del servicio protegido correspondiente; no son el token de consulta del catálogo

Sesión del usuario • Inicio de sesión de Seguridad • Administración desde el microfrontend

Registrar una aplicación propia en Seguridad no concede por sí solo acceso a la API del catálogo. El administrador debe autorizar los recursos del servicio del catálogo que protegerá PEP.

4. Preparar los datos en el catálogo

Antes de emitir el token de consulta:

1. Crear o seleccionar una organización y una aplicación del catálogo. Guardar el UUID de la aplicación.
2. Crear los módulos y funcionalidades que agruparán los parámetros.
3. Crear cada parámetro con su tipo y una funcionalidad de esa aplicación.
4. Crear o seleccionar los ambientes necesarios, por ejemplo desarrollo y producción.
5. En las asociaciones por ambiente, vincular el parámetro al ambiente y elegir un estado activo.
6. Crear sus metadatos seleccionando parámetro, tipo de metadato, ambiente y valor. El ambiente del valor es obligatorio.
7. Comprobar que los registros con estado estén disponibles y que sus padres y fechas de vigencia permitan utilizarlos.

La relación queda así:

```
Organización → Aplicación → Módulo → Funcionalidad → Parámetro
                                                        ├─ Metadato → Ambiente + tipo + valor
                                                        └─ Asociación → Ambiente + estado
```

Un mismo parámetro puede tener valores diferentes por ambiente. Cada metadato vincula el parámetro y el valor con su ambiente mediante idParametro e idAmbiente; esa relación determina en

qué consulta por token aparece el parámetro. La asociación parámetro–ambiente conserva su estado y configuración administrativa, pero no reemplaza el ambiente del metadato.

Los tipos de metadato actuales incluyen JSON, alfanumérico y fecha. El valor debe respetar su tipo. En el editor del microfrontend se escribe JSON: para un texto usar "mi valor"; para un objeto, {"clave":"valor"}. El catálogo valida el contenido en el servidor.

Los metadatos históricos sin ambiente se pueden consultar administrativamente y corregir. No se les asigna un ambiente por inferencia y no se entregan en la consulta por token hasta que se editen para asignárselo.

5. Generar y entregar la credencial de consulta

En el frontend central, abrir Tokens → Nuevo:

1. Seleccionar la aplicación y el ambiente que consumirá el backend.
2. Elegir estado activo y una expiración futura.
3. Crear el token. El frontend genera un secreto aleatorio de 256 bits.
4. Copiar el ID del token, el secreto y el bloque properties mostrado al confirmar.
5. Guardar el secreto en el entorno o gestor de secretos del backend consumidor antes de cerrar la tarjeta.

El servidor guarda el hash del secreto; no se puede recuperar posteriormente desde la lista de tokens. Si se pierde, crear una nueva credencial, actualizar el consumidor y revocar la anterior mediante su estado. Un token para desarrollo no debe reutilizarse como token de producción: cada credencial vincula una aplicación y un ambiente.

No publicar el secreto en Git, archivos servidos por el frontend, environment.ts, localStorage, parámetros de URL, capturas ni logs.

6. Configurar el backend consumidor

Configuración Spring

En application.properties, con los valores privados tomados del entorno:

```
catalogo.parametros.base-url=${CATALOGO_PARAMETROS_BASE_URL}
catalogo.parametros.application-id=${CATALOGO_PARAMETROS_APPLICATION_ID}
catalogo.parametros.token-id=${CATALOGO_PARAMETROS_TOKEN_ID}
catalogo.parametros.secreto=${CATALOGO_PARAMETROS_SECRETO}
```

Ejemplo de URL local al ejecutar Spring en la máquina: http://localhost:8080/catalogo-parametros/api/v1.

Desde un contenedor, localhost identifica ese contenedor. Usar el nombre de servicio de la red Docker o, en este laboratorio con Docker Desktop, http://host.docker.internal:8080/catalogo-parametros/api/v1. En producción usar la URL HTTPS proporcionada por el operador.

Estos nombres de propiedades son una convención del ejemplo: se debe implementar el cliente HTTP que las utiliza. No instalan un SDK ni hacen consultas automáticamente. Spring no carga .env por sí solo; Compose lo carga en el ejemplo. Si se ejecuta Spring directamente, definir las variables en su proceso o en su configuración de despliegue.

Petición de consulta

El backend consumidor realiza:

```
GET /catalogo-parametros/api/v1/tokens/{ID_DEL_TOKEN}/parametros
Accept: application/json
Authorization: Bearer SECRETO_DEL_TOKEN
```

No agregar una aplicación ni un ambiente alternativos a la petición: el token define el alcance. No enviar la credencial PEP en este Bearer.

La respuesta contiene mensajes, aplicacion, ambiente y la lista parametros. Cada elemento incluye parametro, tipoParametro, funcionalidad, modulo, metadatos y configuracionAmbiente. Cada metadato contiene su entidad, incluido idAmbiente, y su tipo. Leer los valores desde parametros[].metadatos[].metadato.valor.

El catálogo filtra los parámetros por aplicación, ambiente, disponibilidad y estado; filtra los metadatos por el ambiente del token. Puede devolver una lista vacía o un parámetro sin valores en ese ambiente. No se debe sustituir esa ausencia por valores de otro ambiente.

Cliente de backend autocontenido para el ejemplo

La entrega incluye ejemplos/consulta-parametros.mjs, ejecutable con Node.js 18+ sin paquetes adicionales. Sirve como referencia de la petición HTTP aunque tu backend use Java u otro lenguaje. Configurar en el entorno privado del proceso:

```
CATALOGO_PARAMETROS_BASE_URL=http://localhost:8080/catalogo-parametros/api/v1
CATALOGO_PARAMETROS_TOKEN_ID=UUID_TOKEN
CATALOGO_PARAMETROS_SECRETO=SECRETO_PRIVADO
CATALOGO_PARAMETROS_APLICACION_ID=UUID_APLICACION
CATALOGO_PARAMETROS_AMBIENTE_ID=UUID_AMBIENTE
CATALOGO_PARAMETROS_PARAMETRO_ID=UUID_PARAMETRO
CATALOGO_PARAMETROS_METADATO_ID=UUID_METADATO
```

Los cuatro últimos UUID identifican el alcance y el valor que se espera leer; no agregan filtros al endpoint. Sustituir los marcadores antes de ejecutar y usar HTTPS para el despliegue publicado. No guardar el secreto en un archivo versionado. Con las variables disponibles en el proceso, desde esta carpeta ejecutar:

```
node ejemplos/consulta-parametros.mjs
```

Resultado esperado para el objeto del ejemplo: {"maximoProductos":100}. El cliente comprueba HTTP, aplicación, ambiente, parámetro, metadato y entero positivo; impone cinco segundos de espera y rechaza redirecciones. Si falta la configuración o el valor, termina con error controlado sin imprimir el secreto ni el cuerpo del proveedor. En producción ajustar límites de respuesta y la política de caché según el volumen y el caso de uso. Este ejemplo no incorpora caché ni reintentos automáticos.

En Spring, el adaptador existente en BackPrueba aplica la misma petición mediante WebClient. Para integrarlo en otro proyecto crear un puerto de consulta, un adaptador HTTP con URL/token/secreto configurables y un DTO que refleje la respuesta. Inyectar el puerto en el caso de uso; comprobar alcance y seleccionar el metadato antes de utilizar valor. No agregar una dependencia Maven de Parámetros: esta integración se realiza por HTTP y el paquete administrativo se distribuye como ESM. El starter de Mensajes de la guía de referencia no es un cliente de Parámetros.

Cliente HTTP y API propia

El ejemplo completo y ejecutable está en BackPrueba. Sus puntos de integración son:

- Propiedades.
- Puerto del cliente.
- Adaptador WebClient.
- Caso de uso.

Este ejemplo limita tiempo y tamaño de respuesta, evita devolver cuerpos de error externos y comprueba que la aplicación recibida coincida con la configurada. Su API propia de consulta entrega datos de negocio a Angular, sin exponer la credencial del catálogo.

En una aplicación real, proteger esa API propia con la autenticación de la aplicación consumidora. No reintentar indefinidamente credenciales inválidas. Si se agrega caché, definir su vigencia e invalidación para no seguir utilizando valores que el catálogo ya desactivó.

7. Autorizar la administración en Seguridad

Antes de montar el microfrontend, el usuario debe tener una sesión válida y permisos sobre los recursos administrativos del catálogo. El UUID del componente no es una credencial.

Sea BASE la ruta /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion:

Método	Recurso que debe registrar y conceder
GET	BASE
GET	BASE/events
POST	BASE/modulos, BASE/funcionalidades, BASE/parametros, BASE/metadatos, BASE/metadatos-ambiente
PUT	Cada una de las cinco rutas anteriores con /{id} al final
DELETE	Cada una de las cinco rutas anteriores con /{id} al final

Registrar las rutas completas, sustituyendo BASE por su prefijo real; conservar los segmentos variables admitidos por Seguridad. Asociar los recursos a un rol o perfil y asignarlo al usuario dentro de la aplicación protegida del catálogo. Registrar el recurso sin concederlo no basta.

Para preparar datos desde el frontend central también se necesitan sus recursos REST y SSE: organizaciones, aplicaciones, ambientes, módulos, funcionalidades, parámetros, metadatos, asociaciones, tipos, estados y tokens según las operaciones que realice ese administrador. Los permisos del microfrontend no conceden automáticamente todos los permisos de la consola central.

El laboratorio incluye un script de referencia para el rol de prueba. No copiar su nombre, usuarios o alcance como configuración de producción: el administrador de cada despliegue debe asignar sus propios permisos. No desactivar PEP ni declarar públicas las rutas administrativas para resolver un 403.

8. Sesión, dominios, HTTPS y CORS

El microfrontend envía la sesión del navegador mediante peticiones con credenciales. El host debe resolver el inicio de sesión y ejecutar su flujo de autenticación cuando reciba onAuthenticationRequired. El widget no crea usuarios, no inicia sesión automáticamente y no recibe secretos de consulta.

En este laboratorio las aplicaciones usan localhost con distintos puertos, por lo que se pudo iniciar sesión desde la consola y reutilizar la cookie. Esto no garantiza compartir sesión entre dominios diferentes en producción. Coordinar con el equipo de Seguridad el BFF, dominio y ruta de cookies, SameSite, Secure, orígenes y destinos de retorno permitidos. Un proxy no hace que una cookie de otro dominio aparezca automáticamente en el navegador.

Usar HTTPS para los dominios publicados. La configuración del widget solo admite HTTP para direcciones de loopback en desarrollo y desde una página HTTP. Rechaza credenciales, query strings y fragmentos en apiBaseUrl.

En el despliegue del catálogo, agregar el origen exacto del host, por ejemplo https://mi-aplicacion.example, a FRONTEND_ALLOWED_ORIGINS. Incluir esquema y puerto cuando corresponda. En el Compose del catálogo se pueden separar varios orígenes con comas; recrear el servicio después de cambiar la configuración. No usar * para un API con credenciales.

El CORS público del archivo JavaScript permite descargar el paquete; no concede permisos sobre los datos ni resuelve la sesión del API.

9. Publicar o consumir el paquete del microfrontend

El operador entrega estos archivos del mismo build:

```
parametros-ui/
  parametros-administration.js
  parametros-administration.d.ts
```

```
  types.d.ts
  manifest.json
  ...avisos de licencias generados
```

En el laboratorio el ESM se sirve desde http://localhost:4200/parametros-ui/parametros-administration.js. Para compilarlo desde el código del proveedor: npm ci y npm run build:microfrontend dentro del frontend central, usando su versión compatible de Node. El resultado queda en dist/parametros-ui; su Dockerfile ya lo publica.

El consumidor puede cargar la URL proporcionada o alojar una copia controlada de esos archivos en su dominio. En ambos casos sigue apuntando al backend central del catálogo. Servir el JS como JavaScript, sin devolver index.html para archivos inexistentes. Conservar los avisos de licencia y actualizar los archivos de distribución juntos.

Si el host usa Content Security Policy, permitir el origen del módulo en sus reglas de scripts y el destino del API/SSE en connect-src. El widget inserta estilos dentro de Shadow DOM; la política de estilos debe admitir el CSS distribuido. La API actual no ofrece una opción para suministrar un nonce de estilos.

10. Montar el componente

Ejemplo independiente del framework, cargado solo al pulsar un botón. Reemplazar el UUID y las URLs por los del despliegue:

```
<button id="abrir-parametros" type="button">Gestionar parámetros</button>
<button id="cerrar-parametros" type="button">Cerrar gestión</button>
<p id="estado-parametros" role="status"></p>
<div id="gestion-parametros"></div>

<script type="module">
  const container = document.querySelector('#gestion-parametros');
  const estado = document.querySelector('#estado-parametros');
  let handle;
  let generation = 0;

  document.querySelector('#abrir-parametros').onclick = async () => {
    const actual = ++generation;
    handle?.unmount();
    handle = undefined;
    estado.textContent = 'Cargando gestión de parámetros…';
    try {
      const remote = await import('/parametros-ui/parametros-administration.js');
      if (actual !== generation) return;
```

```
      handle = remote.mount(container, {
        applicationId: 'UUID_REAL_DE_LA_APLICACION_DEL_CATALOGO',
        apiBaseUrl: location.origin + '/catalogo-parametros/api/v1',
        refreshIntervalMs: 60000,
        onAuthenticationRequired: () => {
          // Aquí el host activa su flujo de inicio de sesión de Seguridad.
          estado.textContent = 'Inicie sesión para administrar los parámetros.';
        },
        onChange: event => {
          // Refrescar la consulta de negocio mediante el backend propio.
          console.log('Cambio de parámetros:', event.resource, event.operation);
        }
      });
      await handle.ready;
      if (actual === generation) estado.textContent = '';
    } catch (error) {
      if (actual === generation) estado.textContent = 'No se pudo abrir la gestión. Revise conexión,
sesión y permisos.';
    }
  };

  document.querySelector('#cerrar-parametros').onclick = () => {
    generation++;
    handle?.unmount();
    handle = undefined;
    estado.textContent = '';
  };
</script>
```

El ejemplo supone que el host publica o enruta /parametros-ui/ y /catalogo-parametros/. Si se usan URLs externas, configurar sesión y CORS según el apartado anterior. El texto UUID_REAL_... es un marcador y debe reemplazarse por el UUID real; el componente lo valida.

Contrato de montaje

Opción • Uso

applicationId • Obligatoria: UUID de la aplicación del catálogo

apiBaseUrl • Obligatoria: raíz del API, incluida /catalogo-parametros/api/v1, sin query ni fragmento

refreshIntervalMs • Opcional: 60000 por defecto; cero desactiva el refresco periódico; de 30000 a 86400000 para otros intervalos

onAuthenticationRequired • Callback del host ante una consulta que recibe 401

onChange • Notifica cambios propios y cambios externos efectivos tras actualizar los datos; no entrega credenciales

theme • Opcional: accent, fontFamily y radius para adaptar el aspecto

mount() retorna un handle con ready, refresh(), unmount() y el elemento montado. Esperar ready para conocer el resultado de la carga inicial. refresh() permite volver a intentar la consulta. Invocar unmount() al cerrar la pantalla o destruir el componente del host, también cuando ready haya fallado. El módulo no usa el router del host y aísla sus estilos con Shadow DOM.

onChange incluye applicationId, resource, operation (created, updated o deleted) e id cuando aplica. Es una notificación para sincronizar vistas, no una bitácora persistente de todos los cambios.

Integración Angular

El ejemplo Angular completo está en FrontPrueba. Revisar:

- Loader ESM: importa la URL en tiempo de ejecución.
- Componente host: crea el montaje al pulsar el botón, espera ready, maneja sesión y llama unmount() al ocultar o destruir.
- Plantilla: contiene el elemento del montaje y la consulta propia.
- Servicio de consulta: llama al backend propio, sin Bearer del catálogo en Angular.

En una aplicación Angular existente, crear un componente contenedor que haga lo mismo: obtener un HTMLElement mediante ViewChild, importar el módulo después de que exista la vista y desmontarlo en ngOnDestroy. Si la importación es asíncrona, ignorar su resultado cuando la vista haya sido destruida. No copiar el paquete Angular del proveedor dentro del árbol de componentes del host: utilizar su API mount().

11. Configurar el proxy y los eventos SSE

Ejemplo Nginx para un host con el API y paquete en rutas del mismo origen. Reemplazar los servidores de ejemplo:

```
location /parametros-ui/ {
    proxy_pass https://catalogo-front.example;
    proxy_set_header Host $proxy_host;
    proxy_ssl_server_name on;
}

location /catalogo-parametros/ {
    proxy_pass https://catalogo-api.example;
```

```
    proxy_set_header Host $proxy_host;
    proxy_ssl_server_name on;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_buffering off;
    proxy_cache off;
    proxy_read_timeout 75s;
}
```

Conservar las rutas y los encabezados de sesión/XSRF que necesite el despliegue. No agregar el secreto de consulta como Authorization a las peticiones administrativas del navegador. No almacenar en caché respuestas administrativas ni el flujo de eventos.

El widget escucha GET .../administracion/events, evento catalogo-aplicacion. El servidor envía latidos cada 15 segundos y filtra las notificaciones por la aplicación; también notifica cambios de ambientes auxiliares. No envía valores de metadatos en esas notificaciones: el cliente vuelve a consultar su rama autorizada.

El cliente agrupa ráfagas durante 250 ms, reconecta con espera creciente hasta 30 segundos y conserva un refresco periódico como respaldo. No sobrescribe formularios abiertos: aplica los cambios pendientes al cerrarlos. SSE sigue activo aunque refreshIntervalMs sea cero. El frontend central también escucha los eventos, de modo que una edición se refleja en ambas interfaces sin recargar.

12. Vigencia, estados y fechas

• Enviar las fechas y horas con zona ISO-8601; el catálogo normaliza los instantes a UTC. Los formularios muestran hora local y conservan el instante al guardar.

• El instante final de la vigencia ya no permite utilizar el registro.

• La tarea automática revisa vencimientos cada minuto; la consulta también comprueba disponibilidad, por lo que no se debe asumir que un registro vencido sigue siendo utilizable hasta el siguiente ciclo.

• La desactivación manual o por vencimiento de un padre propaga los estados existentes de sus dependientes. No se agregan estados a entidades que no los tienen.

• La reactivación es individual, con padres disponibles y vigencia válida; no reactiva toda una rama automáticamente.

Consultar el detalle en Vigencias y estados.

13. Verificación antes de entregar la integración

1. Con el backend consumidor configurado, consultar sus parámetros y comprobar aplicación y ambiente recibidos.

2. Crear valores diferentes del mismo parámetro en desarrollo y producción. Confirmar que cada token recibe solo los valores de su ambiente.

3. Montar el microfrontend con sesión autorizada y comprobar carga, selectores y creación con ambiente obligatorio.

4. Editar un metadato desde el widget y comprobar su actualización en el frontend central y en la consulta propia.

5. Editarlo desde el frontend central y comprobar el cambio por SSE en el widget y la consulta propia, sin recargar.

6. Probar un usuario sin permisos y una sesión ausente: la administración debe rechazarse. Probar un token inválido, inactivo o vencido: la consulta debe rechazarse.

7. Desactivar un parámetro o su padre y comprobar que la consulta deja de entregarlo.

8. Abrir y cerrar la gestión varias veces: no deben quedar widgets, conexiones o temporizadores duplicados.

9. Revisar que no se exponga el secreto en el bundle, almacenamiento del navegador, URLs ni respuestas de configuración.

El repositorio incluye un laboratorio para comprobar la consulta desde Spring, el aislamiento por ambiente y la sincronización SSE en ambos sentidos. Las pruebas de este apartado deben ejecutarse en la instalación del equipo consumidor. El script probar-integracion-real.mjs permite repetir esa comprobación con la cuenta administradora de su entorno. Sus credenciales se suministran por variables de entorno; no copiarlas a este documento ni al código.

14. Problemas frecuentes

Síntoma • Qué revisar

No carga el módulo o aparece un error de MIME • URL del JS, publicación de archivos, tipo JavaScript, CORS del módulo y reglas del proxy; no debe devolver el HTML de la SPA

applicationId inválido o 404 administrativo • Usar el UUID de Aplicaciones del catálogo, no el ID de Seguridad ni el token

401 administrativo • Sesión y cookies disponibles para el API; ejecutar el flujo de inicio de sesión del host

403 administrativo • Origen permitido y recursos, rol/perfil, usuario y ambiente autorizados en Seguridad

Funciona consultar, pero no crear o editar • Permisos POST/PUT/DELETE correspondientes; sesión/XSRF y ambiente obligatorio del metadato

Consulta vacía • Aplicación y ambiente del token, metadatos asignados a ese ambiente, estados y vigencias de padres

Parámetro sin valores • Metadatos asignados al ambiente del token; los históricos sin ambiente se excluyen

No actualiza en tiempo real • GET /events autorizado, respuesta SSE, proxy sin buffering/caché y tiempo de espera superior a los latidos

Error de contenido mixto • Host y API publicados con HTTPS; no usar un API HTTP desde una página HTTPS

Token rechazado • ID y secreto emparejados, estado activo y expiración futura; comprobar que no se está usando la credencial PEP

El ejemplo Spring devuelve 424/502/503 • 424: credencial rechazada/dependencia fallida; 502: conexión/respuesta/alcance externo inválidos; 503: configuración pendiente. Son códigos del consumidor de ejemplo, no una traducción universal del API del catálogo

15. Referencias y prueba local rápida

- Contrato administrativo OpenAPI.
- Contrato general OpenAPI.
- Subcontrato de consulta por token usado por el ejemplo.
- Contrato TypeScript del microfrontend.
- Guía breve del microfrontend.

Con el catálogo y Seguridad funcionando, desde BackPrueba:

```
docker compose up -d --build
```

Abrir http://localhost:4300, pulsar Consultar parámetros y luego Administrar parámetros. El ejemplo local ya preparado usa su propia aplicación y credencial de consulta. Para otra instalación, generar una nueva credencial y completar BackPrueba/.env o ejecutar configurar.ps1 siguiendo su README.

Para detener solo el consumidor de prueba, ejecutar docker compose down desde BackPrueba. No elimina ni detiene los datos y servicios del catálogo central.

Anexo A. Levantar el proveedor en este workspace

Este anexo es para quien operará una instancia local. Si tu equipo consume el catálogo central ya desplegado, utiliza sus URL y continúa con la preparación de datos del apartado 4.

A.1 Preparar Seguridad

1. Levantar el stack de Seguridad/security-baseline-local según su manual. La integración documentada utiliza el stack 0.1.6 y el starter 0.1.3; son versiones diferentes.

2. En Seguridad, registrar el servicio del catálogo con nombre Catalogo y URL local http://localhost:8080. Conservar el nombre exacto o actualizar de forma consistente la configuración del servicio.

3. Guardar la credencial técnica entregada por Seguridad como PEP_REGISTRATION_TOKEN, solo en el backend del catálogo.

4. Seguir el paso 0. Permitir la pantalla de Parámetros en Seguridad, al inicio de esta guía: configurar las dos variables PDP, comprobar su interpolación en Compose y aplicar el cambio. Conservar los orígenes existentes.

5. Registrar los recursos, concederlos a un rol y asignar ese rol al usuario que administrará el catálogo. La lista del anexo B corresponde al widget; la consola central requiere además los recursos de sus pantallas.

Comprobación: poder iniciar sesión y regresar al origen permitido. Registrar el servicio en Seguridad no crea la aplicación de negocio dentro de Parámetros.

A.2 Configurar y arrancar

El backend actual requiere Java 25 para ejecución directa, SurrealDB y acceso al Azure Key Vault configurado. Docker Compose compila el stack y espera las carpetas hermanas de frontend y Security UI de este workspace. No se distribuye aquí un instalador independiente equivalente al ZIP de Mensajes.

Configurar las credenciales Azure, las propiedades de SurrealDB y los secretos Maven según el README del backend. Inyectarlos desde el entorno o el gestor de secretos del despliegue. La configuración actual usa Azure Key Vault; no presupone el proyecto Doppler de Mensajes.

Variables de Seguridad del proveedor:

Variable • Ejemplo local o uso

SECURITY_APPLICATION_NAME • Catalogo, igual al registro de Seguridad

SECURITY_ENVIRONMENT • local

PEP_REGISTRATION_TOKEN • Credencial técnica del servicio en Seguridad

PEP_URL • http://host.docker.internal:18081 desde Docker Desktop

PEP_ALLOW_INSECURE_HTTP • true solo en laboratorio HTTP; false en producción

FRONTEND_ALLOWED_ORIGINS • http://localhost:4200,http://localhost:4300

Desde la raíz del workspace:

```
cd CatalogoParametros/CatalogoParametrosUcoLab
docker compose up ––build –d
```

Comprobación: abrir http://localhost:8080/actuator/health, la consola en http://localhost:4200 y Swagger en http://localhost:8080/swagger-ui.html. Si la salud falla, revisar los logs y las dependencias antes de preparar datos. El host de prueba usa 4300: ejecutar solo uno de los ejemplos que requieran ese puerto.

Para apagar el proveedor conservando sus volúmenes:

```
docker compose down
```

Anexo B. Rutas del microfrontend y consulta

Prefijo administrativo: /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion. En la tabla, BASE representa ese prefijo completo. Son 17 operaciones administrativas para el widget.

Método • Ruta

GET • BASE

GET • BASE/events

POST • BASE/modulos

PUT • BASE/modulos/{id}

DELETE • BASE/modulos/{id}

POST • BASE/funcionalidades

PUT • BASE/funcionalidades/{id}

DELETE • BASE/funcionalidades/{id}

POST • BASE/parametros

PUT • BASE/parametros/{id}

DELETE • BASE/parametros/{id}

POST • BASE/metadatos

PUT • BASE/metadatos/{id}

DELETE • BASE/metadatos/{id}

POST • BASE/metadatos-ambiente

PUT • BASE/metadatos-ambiente/{id}

DELETE • BASE/metadatos-ambiente/{id}

La consulta entre servidores usa GET /catalogo-parametros/api/v1/tokens/{id}/parametros. Esta ruta omite PEP y valida su propio Bearer, estado y vencimiento. No exige registrar cada token en Seguridad. La creación, listado y administración de tokens sí pasan por Seguridad. El contrato general y la colección Postman cubren el resto del catálogo.

Anexo C. Leer el valor en tu lógica de negocio

La respuesta tiene esta estructura conceptual (se omiten campos de las entidades):

```
mensajes: lista de mensajes de respuesta
aplicacion: aplicación autorizada
ambiente: ambiente autorizado
parametros[]
  parametro: definición e ID
  tipoParametro: tipo del parámetro
  funcionalidad: agrupación funcional
  modulo: agrupación del módulo
  metadatos[]
    metadato: id, idParametro, idTipoMetadato, idAmbiente, valor
    tipoMetadato: tipo del valor
  configuracionAmbiente: asociación y estado del ambiente
```

Para el ejemplo Inventario:

1. Ejecutar la consulta por token desde el adaptador HTTP del backend.

2. Comprobar que aplicacion.id coincide con el UUID configurado y que el ambiente es el esperado.

3. Localizar el parámetro por su UUID configurado. Si se usa el nombre, definir su contexto de módulo y funcionalidad para evitar ambigüedad.

4. Seleccionar el metadato acordado por ID o tipo; un parámetro puede tener varios metadatos. No asumir que el primer elemento siempre contiene el valor buscado.

5. Leer metadato.valor.maximoProductos y validar que sea un entero positivo antes de aplicarlo.

6. Entregar al frontend propio únicamente los datos que necesita la pantalla.

Si el catálogo falla: definir la política del parámetro según su uso. Para una configuración obligatoria, devolver un error controlado e impedir la operación que depende de ella. Para una configuración opcional, usar un valor de respaldo aprobado y registrar el fallo sin secretos. Una caché debe tener una vigencia explícita; la consulta no incluye un mecanismo automático para actualizar la caché privada de cada backend.

Comprobación: cambiar 100 a 120 en Desarrollo y volver a consultar desde el backend. El valor usado por la lógica debe ser 120; Producción debe conservar su propio valor. Si el consumidor implementa caché, comprobarlo después de su invalidación o vencimiento.

Anexo D. Lista de entrega del equipo consumidor

- URL del API y del paquete confirmadas para el ambiente.
- UUID de aplicación y de ambiente documentados; credencial guardada de forma privada.
- Consulta por token funcionando desde la red real del backend.
- Sesión, retorno de login y permisos administrativos comprobados.
- Valor por ambiente y manejo de ausencias definidos.
- Montaje y desmontaje del widget funcionando.
- Sincronización central/widget y refresco de la consulta propia verificados.
- Errores de token, sesión, permisos y conectividad probados.

Documento elaborado a partir del código y documentación presentes en el workspace al 3 de noviembre de 2026. Los ejemplos son de laboratorio y no contienen credenciales reales. La elaboración de esta guía no ejecuta ni certifica el despliegue de cada equipo.

Anexo E. Referencia de cuerpos y comportamiento HTTP

Todas las rutas de negocio parten de /catalogo-parametros/api/v1. Los ejemplos son ilustrativos y usan marcadores que se sustituyen por UUID reales. Content-Type: application/json para cuerpos; Accept: application/json para respuestas, excepto SSE. PUT recibe todos los campos del recurso. El

único PATCH implementado es el cambio de estado del token.

Cuerpos de la consola central

POST crea sobre la colección; PUT actualiza sobre /{id} con el mismo cuerpo. Para organizaciones, aplicaciones, módulos y funcionalidades las fechas del API central son opcionales; si se envían ambas, el fin no puede preceder al inicio. Los nombres de estos recursos y parámetros se validan entre 3 y 50 caracteres. Los IDs referenciados deben existir.

POST /organizaciones y PUT /organizaciones/{id}

```
{
  "nombre": "UCO",
  "fechaInicio": "2026-11-01T00:00:00-05:00",
  "fechaFinal": "2027-12-31T23:59:59-05:00"
}
```

POST /aplicaciones y PUT /aplicaciones/{id}

```
{
  "nombre": "Inventario UCO",
  "idOrganizacion": "{{idOrganizacion}}",
  "activa": "true",
  "fechaInicio": "2026-11-01T00:00:00-05:00",
  "fechaFinal": "2027-12-31T23:59:59-05:00"
}
```

POST /modulos y PUT /modulos/{id}

```
{
  "nombre": "Inventario",
  "activo": "true",
  "fechaInicio": "2026-11-01T00:00:00-05:00",
  "fechaFinal": "2027-12-31T23:59:59-05:00",
  "idAplicacion": "{{idAplicacion}}"
}
```

POST /funcionalidades y PUT /funcionalidades/{id}

```
{
  "nombre": "Control de existencias",
  "idModulo": "{{idModulo}}",
  "activo": "true",
  "fechaInicio": "2026-11-01T00:00:00-05:00",
  "fechaFinal": "2027-12-31T23:59:59-05:00"
}
```

POST /parametros y PUT /parametros/{id}

```
{
  "nombre": "configuracion-inventario",
  "idFuncionalidad": "{{idFuncionalidad}}",
  "idTipoParametro": "{{idTipoParametro}}",
  "activo": "true"
}
```

POST /metadatos y PUT /metadatos/{id}

```
{
  "idParametro": "{{idParametro}}",
  "idTipoMetadato": "{{idTipoMetadato}}",
  "idAmbiente": "{{idAmbiente}}",
  "valor": {
    "maximoProductos": 100
  }
}
```

POST /ambientes y PUT /ambientes/{id}

```
{
  "nombre": "Desarrollo"
}
```

POST /estados-ambiente y PUT /estados-ambiente/{id}

```
{
  "nombre": "Activo"
}
```

POST /estados-metadato-ambiente y PUT /estados-metadato-ambiente/{id}

```
{
  "nombre": "Activo"
}
```

POST /metadatos-ambiente y PUT /metadatos-ambiente/{id}

```
{
  "idParametro": "{{idParametro}}",
  "idAmbiente": "{{idAmbiente}}",
  "idEstadoMetadatoAmbiente": "{{idEstadoMetadatoAmbiente}}"
}
```

activo y activa del API central de crear/actualizar se envían como cadenas; en las respuestas son booleanos. Para cambiar el estado de aplicaciones, módulos, funcionalidades y parámetros, usar POST /{recurso}/{id}/cambiarestado con {"activo": false} (booleano, incluso para aplicaciones). Los tipos de parámetro, tipos de metadato y estados de token solo tienen consulta.

El campo idAmbiente del metadato es obligatorio para nuevos valores y actualizaciones. valor es un valor JSON, cuyo contenido debe ser compatible con el tipo de metadato existente. Para el ejemplo del objeto elegir el tipo JSON. Consultar el catálogo de tipos antes de enviar. El tipo json exige objeto o arreglo; alfanumerico exige cadena no vacía; date exige una cadena de fecha yyyy-MM-dd. No asumir que un número suelto se acepta como tipo JSON. La asociación metadatos-ambiente no contiene valor ni tipo de metadato. estados-ambiente es un catálogo independiente: el DTO de ambiente solo tiene nombre.

Cuerpos de administración embebida

Usar POST .../aplicaciones/{idAplicacion}/administracion/{recurso} y PUT en esa misma ruta con /{id}. El ID de aplicación se obtiene de la ruta; el servidor comprueba que el recurso actual y el padre propuesto pertenecen a esa aplicación. Una referencia ajena se rechaza.

Aquí activo es booleano. En módulos y funcionalidades son obligatorias ambas fechas, a diferencia del API central. Omitir activo se interpreta como verdadero; para desactivar enviarlo explícitamente como falso.

Cuerpo embebido: modulos

```
{
  "nombre": "Inventario",
  "activo": true,
  "fechaInicio": "2026-11-01T00:00:00-05:00",
  "fechaFinal": "2027-12-31T23:59:59-05:00"
}
```

Cuerpo embebido: funcionalidades

```
{
  "nombre": "Control de existencias",
  "idModulo": "{{idModulo}}",
  "activo": true,
  "fechaInicio": "2026-11-01T00:00:00-05:00",
  "fechaFinal": "2027-12-31T23:59:59-05:00"
}
```

Cuerpo embebido: parametros

```
{
  "nombre": "configuracion-inventario",
  "idFuncionalidad": "{{idFuncionalidad}}",
  "idTipoParametro": "{{idTipoParametro}}",
  "activo": true
}
```

Cuerpo embebido: metadatos

```
{
  "idParametro": "{{idParametro}}",
  "idTipoMetadato": "{{idTipoMetadato}}",
  "idAmbiente": "{{idAmbiente}}",
  "valor": {
    "maximoProductos": 100
  }
}
```

Cuerpo embebido: metadatos-ambiente

```
{
```

```
  "idParametro": "{{idParametro}}",
  "idAmbiente": "{{idAmbiente}}",
  "idEstadoMetadatoAmbiente": "{{idEstadoMetadatoAmbiente}}"
}
```

Las escrituras embebidas responden 201 al crear y 200 al actualizar/eliminar, con {"mensajes": ["Registro creado."]} o el mensaje de la operación. Consultar de nuevo la base administrativa para obtener los datos y los IDs. Su GET responde { "mensajes": [], "catalogo": { ... } }, con aplicación, módulos, funcionalidades, parámetros, metadatos, asociaciones y catálogos auxiliares. No usar el listado administrativo como respuesta de runtime: contiene registros inactivos para poder gestionarlos.

Crear, consultar, validar y revocar tokens

POST /tokens (requiere permisos administrativos):

```
{
  "token": "SECRETO_ALEATORIO_GENERADO_POR_EL_CLIENTE",
  "estadoToken": "UUID_ESTADO_ACTIVO",
  "aplicacionId": "UUID_APLICACION_CATALOGO",
  "ambienteId": "UUID_AMBIENTE",
  "fechaExpiracion": "2027-12-31T23:59:59Z"
}
```

La consola genera el secreto y lo muestra al crear; si se llama directamente al API, el cliente debe generar y conservar token. El servidor guarda un hash BCrypt y no devuelve el secreto. La respuesta 201 contiene mensajes y tokens, con ID y datos públicos. Guardar el ID retornado y el secreto original en el backend consumidor. estadoToken es el UUID obtenido en GET /estados-token, no el texto "Activo".

GET /tokens y GET /estados-token no tienen paginación. GET /tokens/{id} y GET /estados-token/{id} consultan uno. PATCH /tokens/{id}/estado recibe {"estadoToken":"UUID_ESTADO"}; seleccionar el estado de revocación/inactividad existente en el catálogo. POST /tokens/{id}/validar recibe {"token":"SECRETO_ORIGINAL"} y responde con mensajes y valido; un secreto inválido produce 401. Esa validación administrativa sigue pasando por PEP. La consulta de runtime ya valida la credencial, así que el consumidor no necesita llamar a /validar antes de cada lectura.

Respuesta de consulta por token

GET /tokens/{id}/parametros recibe Authorization: Bearer SECRETO_ORIGINAL. Ejemplo parcial de una respuesta 200; se omiten propiedades para mostrar dónde está el valor:

```
{
  "mensajes": [],
  "aplicacion": { "id": "UUID_APLICACION", "nombre": "Inventario UCO" },
  "ambiente": { "id": "UUID_AMBIENTE", "nombre": "Desarrollo" },
  "parametros": [
    {
      "parametro": { "id": "UUID_PARAMETRO", "nombre": "configuracion-inventario", "activo": true },
      "tipoParametro": { "id": "UUID_TIPO_PARAMETRO" },
      "funcionalidad": { "id": "UUID_FUNCIONALIDAD" },
      "modulo": { "id": "UUID_MODULO" },
      "metadatos": [
        {
          "metadato": {
            "id": "UUID_METADATO", "idParametro": "UUID_PARAMETRO",
            "idTipoMetadato": "UUID_TIPO_JSON", "idAmbiente": "UUID_AMBIENTE",
            "valor": { "maximoProductos": 100 }
          },
          "tipoMetadato": { "id": "UUID_TIPO_JSON", "nombre": "JSON" }
        }
      ],
      "configuracionAmbiente": null
    }
  ]
}
```


configuracionAmbiente puede ser nula; cuando existe contiene la asociación y su estado. No contiene el valor del parámetro. Puede haber varios metadatos por parámetro: seleccionar el acordado por ID/tipo, no el primero por posición. Una respuesta 200 con lista vacía es válida y debe manejarse según la política del consumidor. Un 401 significa que falta el secreto o que es inválido, está inactivo/revocado o vencido.

Listados, respuestas y errores generales

Las colecciones centrales, salvo metadatos, tipos, tokens y estados de token, usan page=1&pageSize=10. Valores menores que uno se normalizan a uno. No hay total ni totalPages: recorrer hasta una página vacía. GET /metadatos acepta idParametro y no pagina. No asumir filtros de relaciones que el controlador no declara.

Las consultas centrales devuelven mensajes y una lista, incluso por ID: organizaciones, aplicaciones, modulos, funcionalidades, parametros, metadatos, tiposParametro, tiposMetadato, ambientes, estadosAmbiente, estadosMetadatoAmbiente, metadatosAmbiente, tokens o estadosToken.

Las escrituras originales de organización, aplicación, módulo, funcionalidad, parámetro y metadato

no devuelven la entidad creada: consultar para recuperar el ID. Las escrituras de módulo y funcionalidad usan actualmente parametros: []; sus consultas sí usan su colección propia. Ambientes, sus estados y asociaciones devuelven la entidad al crear/actualizar. DELETE devuelve 200 con cuerpo, no 204.

HTTP • Interpretación y actuación

200 / 201 • Operación exitosa; leer el cuerpo según el recurso

400 • Cuerpo, UUID, fechas o validación inválidos; corregir los datos

401 • Administración: recuperar sesión. Consulta por token: revisar la credencial privada

403 • Revisar rol, recurso, método, aplicación y ambiente concedidos

404 • Revisar recurso, referencia y pertenencia a la aplicación

409 • Resolver duplicados o relaciones que impiden la operación

500 • Error técnico del catálogo; registrar contexto sin secretos

503 • Seguridad no disponible; recuperar la dependencia antes de reintentar

Los errores globales usan {"mensajes":["Descripción"]} y algunos controladores agregan una colección vacía. La autorización puede responder con el formato de Seguridad. Usar el estado HTTP para clasificar el fallo y tolerar cuerpos distintos, incluidos errores del proxy. No decidir por el texto literal de un mensaje. No hay clave de idempotencia: ante timeout de creación, comprobar si se guardó antes de repetir. Nunca repetir indiscriminadamente escrituras.

Eventos generales y administrativos

Los diez recursos generales con escritura exponen GET /events. Los nombres son organizacion, aplicacion, modulo, funcionalidad, parametro, metadato, ambiente, estadoambiente, estadometadatoambiente y metadatoambiente. El JSON de data lleva el recurso y event con CREATED, UPDATED o DELETED. Usar addEventListener con el nombre; onmessage no recibe eventos nombrados. Los tipos y tokens no tienen SSE general.

El flujo administrativo es GET /aplicaciones/{idAplicacion}/administracion/events, evento catalogo-aplicacion, con latidos cada 15 segundos y notificaciones para volver a consultar. No es una cola durable ni ofrece reanudación con Last-Event-ID. Los publishers generales guardan hasta 100 eventos por publisher en memoria; puede haber repeticiones y no hay orden global garantizado. Tras reconectar, reconciliar con GET. Cerrar conexiones al destruir la vista.

Anexo F. Inventario completo de las 93 operaciones de negocio

La columna Seguridad indica si la operación está incluida en recursos-parametros.xlsx. Se muestran las rutas completas sin host; las 17 del microfrontend ya están incluidas en las 93.

ambientes

• GET /catalogo-parametros/api/v1/ambientes/events — Eventos SSE. Seguridad: SI.

• POST /catalogo-parametros/api/v1/ambientes — Crear. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/ambientes/{id} — Actualizar. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/ambientes/{id} — Eliminar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/ambientes — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/ambientes/{id} — Consultar. Seguridad: SI.

aplicaciones

• GET /catalogo-parametros/api/v1/aplicaciones/events — Eventos SSE. Seguridad: SI.

• GET /catalogo-parametros/api/v1/aplicaciones — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/aplicaciones/{id} — Consultar. Seguridad: SI.

• POST /catalogo-parametros/api/v1/aplicaciones — Crear. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/aplicaciones/{id} — Actualizar. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/aplicaciones/{id} — Eliminar. Seguridad: SI.

• POST /catalogo-parametros/api/v1/aplicaciones/{id}/cambiarestado — Cambiar estado. Seguridad: SI.

Administración embebida

• GET /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/events — Eventos SSE. Seguridad: SI.

• GET /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion — Consultar. Seguridad: SI.

• POST /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/modulos — Crear. Seguridad: SI.

• POST /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/funcionalidades — Crear. Seguridad: SI.

• POST /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/parametros — Crear. Seguridad: SI.

• POST /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/metadatos — Crear. Seguridad: SI.

• POST /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/metadatos-ambiente — Crear. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/modulos/{id} — Actualizar. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/funcionalidades/{id} — Actualizar. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/parametros/{id} — Actualizar. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/metadatos/{id} — Actualizar. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/metadatos-ambiente/{id} — Actualizar. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/modulos/{id} — Eliminar. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/funcionalidades/{id} — Eliminar. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/parametros/{id} — Eliminar. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/metadatos/{id} — Eliminar. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/aplicaciones/{idAplicacion}/administracion/metadatos-ambiente/{id} — Eliminar. Seguridad: SI.

estados-ambiente

• GET /catalogo-parametros/api/v1/estados-ambiente/events — Eventos SSE. Seguridad: SI.

• POST /catalogo-parametros/api/v1/estados-ambiente — Crear. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/estados-ambiente/{id} — Actualizar. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/estados-ambiente/{id} — Eliminar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/estados-ambiente — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/estados-ambiente/{id} — Consultar. Seguridad: SI.

estados-metadato-ambiente

• GET /catalogo-parametros/api/v1/estados-metadato-ambiente/events — Eventos SSE. Seguridad: SI.

• POST /catalogo-parametros/api/v1/estados-metadato-ambiente — Crear. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/estados-metadato-ambiente/{id} — Actualizar. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/estados-metadato-ambiente/{id} — Eliminar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/estados-metadato-ambiente — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/estados-metadato-ambiente/{id} — Consultar. Seguridad: SI.

estados-token

• GET /catalogo-parametros/api/v1/estados-token — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/estados-token/{id} — Consultar. Seguridad: SI.

funcionalidades

• GET /catalogo-parametros/api/v1/funcionalidades/events — Eventos SSE. Seguridad: SI.

• POST /catalogo-parametros/api/v1/funcionalidades — Crear. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/funcionalidades/{id} — Actualizar. Seguridad: SI.

• POST /catalogo-parametros/api/v1/funcionalidades/{id}/cambiarestado — Cambiar estado. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/funcionalidades/{id} — Eliminar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/funcionalidades — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/funcionalidades/{id} — Consultar. Seguridad: SI.

metadatos

• GET /catalogo-parametros/api/v1/metadatos/events — Eventos SSE. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/metadatos/{id} — Actualizar. Seguridad: SI.

• POST /catalogo-parametros/api/v1/metadatos — Crear. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/metadatos/{id} — Eliminar. Seguridad: SI.

- GET /catalogo-parametros/api/v1/metadatos — Consultar. Seguridad: SI.
- GET /catalogo-parametros/api/v1/metadatos/{id} — Consultar. Seguridad: SI.

metadatos-ambiente

- GET /catalogo-parametros/api/v1/metadatos-ambiente/events — Eventos SSE. Seguridad: SI.
- POST /catalogo-parametros/api/v1/metadatos-ambiente — Crear. Seguridad: SI.
- PUT /catalogo-parametros/api/v1/metadatos-ambiente/{id} — Actualizar. Seguridad: SI.
- DELETE /catalogo-parametros/api/v1/metadatos-ambiente/{id} — Eliminar. Seguridad: SI.
- GET /catalogo-parametros/api/v1/metadatos-ambiente — Consultar. Seguridad: SI.
- GET /catalogo-parametros/api/v1/metadatos-ambiente/{id} — Consultar. Seguridad: SI.

modulos

- GET /catalogo-parametros/api/v1/modulos/events — Eventos SSE. Seguridad: SI.
- DELETE /catalogo-parametros/api/v1/modulos/{id} — Eliminar. Seguridad: SI.
- POST /catalogo-parametros/api/v1/modulos — Crear. Seguridad: SI.
- PUT /catalogo-parametros/api/v1/modulos/{id} — Actualizar. Seguridad: SI.
- POST /catalogo-parametros/api/v1/modulos/{id}/cambiarestado — Cambiar estado. Seguridad: SI.
- GET /catalogo-parametros/api/v1/modulos — Consultar. Seguridad: SI.
- GET /catalogo-parametros/api/v1/modulos/{id} — Consultar. Seguridad: SI.

organizaciones

- GET /catalogo-parametros/api/v1/organizaciones/events — Eventos SSE. Seguridad: SI.
- POST /catalogo-parametros/api/v1/organizaciones — Crear. Seguridad: SI.
- PUT /catalogo-parametros/api/v1/organizaciones/{id} — Actualizar. Seguridad: SI.
- DELETE /catalogo-parametros/api/v1/organizaciones/{id} — Eliminar. Seguridad: SI.
- GET /catalogo-parametros/api/v1/organizaciones — Consultar. Seguridad: SI.
- GET /catalogo-parametros/api/v1/organizaciones/{id} — Consultar. Seguridad: SI.

parametros

• GET /catalogo-parametros/api/v1/parametros/events — Eventos SSE. Seguridad: SI.

• POST /catalogo-parametros/api/v1/parametros — Crear. Seguridad: SI.

• PUT /catalogo-parametros/api/v1/parametros/{id} — Actualizar. Seguridad: SI.

• POST /catalogo-parametros/api/v1/parametros/{id}/cambiarestado — Cambiar estado. Seguridad: SI.

• DELETE /catalogo-parametros/api/v1/parametros/{id} — Eliminar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/parametros — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/parametros/{id} — Consultar. Seguridad: SI.

tipos-metadato

• GET /catalogo-parametros/api/v1/tipos-metadato — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/tipos-metadato/{id} — Consultar. Seguridad: SI.

tipos-parametro

• GET /catalogo-parametros/api/v1/tipos-parametro — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/tipos-parametro/{id} — Consultar. Seguridad: SI.

tokens

• POST /catalogo-parametros/api/v1/tokens — Crear. Seguridad: SI.

• GET /catalogo-parametros/api/v1/tokens — Consultar. Seguridad: SI.

• GET /catalogo-parametros/api/v1/tokens/{id} — Consultar. Seguridad: SI.

• PATCH /catalogo-parametros/api/v1/tokens/{id}/estado — Cambiar estado. Seguridad: SI.

• POST /catalogo-parametros/api/v1/tokens/{id}/validar — Validar secreto del token. Seguridad: SI.

• GET /catalogo-parametros/api/v1/tokens/{id}/parametros — Consultar parámetros autorizados. Seguridad: NO.

Anexo G. Rutas técnicas del despliegue

Estas rutas usan el host directamente. No son operaciones de negocio ni están en el Excel de permisos funcionales. Solo /actuator/health está expresamente excluida del PEP en la configuración actual; las demás deben revisarse con el operador y Seguridad según su uso y exposición.

Método	Ruta	Uso
GET	/actuator	Enlaces de gestión
GET	/actuator/health	Salud del servicio
GET	/actuator/info	Información técnica
GET	/actuator/prometheus	Métricas
GET	/v3/api-docs	OpenAPI JSON generado desde el servicio
GET	/v3/api-docs.yaml	OpenAPI YAML generado
GET	/v3/api-docs/swagger-config	Configuración Swagger
GET	/swagger-ui.html	Entrada de Swagger UI
GET	/openapi/catalogo-parametros-v1.yaml	Contrato general estático

Los contratos estáticos y la colección anterior a esta entrega mantienen referencias a 68 operaciones originales y omiten parte de los cambios recientes. Para esta entrega el inventario se extrajo de los 15 controladores actuales y se expandieron los cinco recursos administrativos válidos: 76 operaciones generales y 17 embebidas. Para obtener esquemas del despliegue efectivo, consultar /v3/api-docs y contrastarlo con su versión del servicio.

La validación de esta entrega es documental y estructural: rutas contra controladores, unicidad método/ruta, formato del Excel oficial y cobertura de Postman. No se inició el stack ni se efectuaron escrituras, importaciones o pruebas de autenticación contra servicios reales.